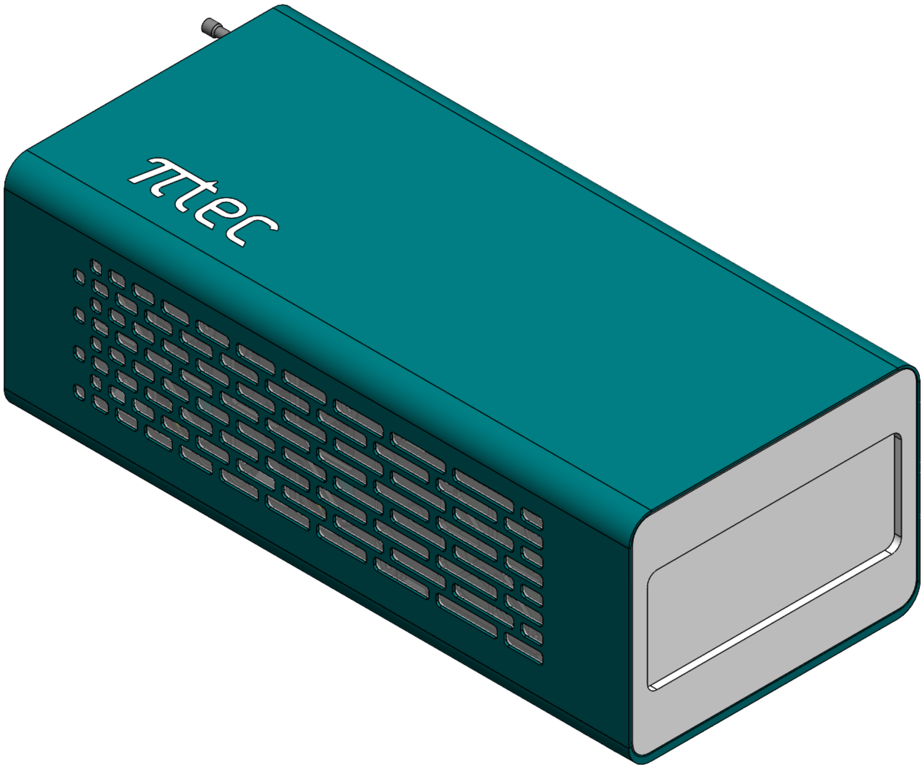
# Compact X-ray Detector

## RAD 800k – Si or CdTe

# Operation User Manual – Software Control



| Rev | Origination Date | Change Description | Originator(s) |
|---|---|---|---|
| 1.0 | 2024-01-03 | Initial Release | Leticia Rosa, Victor Araujo |
| 1.1 | 2024-09-08 | Adjustments in Operation procedures<br>Trigger Procedures | Victor Araujo, Gabriel Couto, Hadassa Gomes |
| 1.2 | 2025-08-10 | Adjusts in Operation procedures | Victor Araujo, Maria Medeiros |

# Table of Content

# 1. Introduction

**RAD 800 k** is a compact area detector for X-Ray applications based on the latest noise-free, photon counting hybrid-pixel technology (Figure 1). It is made two arrays of six Medipix3RX ASICs of 55 $\mu m^2$ pixel size, resulting in **786 kPixels** of resolution, with an active area of **33.2 x 87.3 mm²**. This detector offers high resolution, high speed and high efficiency in a small form factor – suitable for several techniques and applications required for synchrotrons.

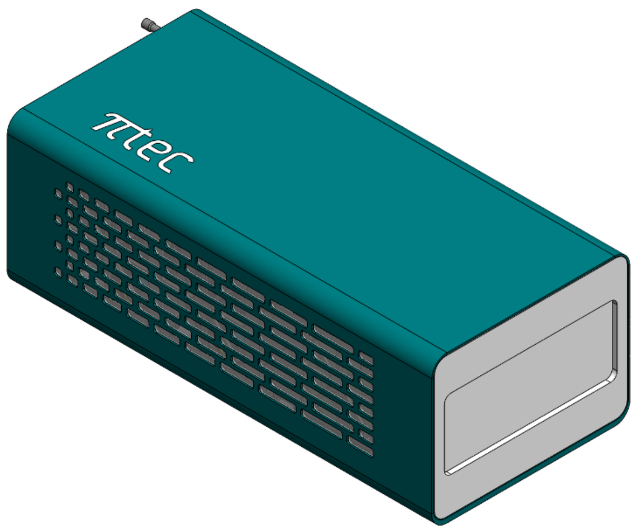



**Figure 1:** Compact Area Detector - RAD.

# 2. System Overview

The compact X-ray detector comprises a front-end and a backend that are synchronized over a high-speed link. The front-end is composed of hardware and firmware elements, and the communication with the backend occurs using Remote Direct Memory Access over Converged Ethernet version 1 (RoCEv1).

The back-end comprises a server hardware and a high-performance software. Its job is to process the received photon counting information to generate a raw image and then apply image processing algorithms to restore the final image. The higher the server hardware specification, the higher is the number of images that can be obtained within a given time.

Throughout this document, the term detector software is used to refer to the Compact Detector complete software stack called Pimega Software Suite (PSS). A software development kit (SDK) is available on the PSS that includes an application programming interface (API), a functions library, and a driver for communication with the detector hardware and the backend server. The user can configure and operate the detector using a Command Line Interface (CLI).

The compact X-ray detector software system can be controlled primarily through the Command Line Instruction (CLI). In this manual we present the software features and basic setting routines for operating the detector properly. The software suit must be correctly installed in the computer connected with the detector. For further information, read the Software Installation Document.

# 3. CLI Control System

## 3.1. Initializing Services

To start running the detector, ensure the software is correctly installed (consult Software Installation Guide if necessary) and the connection between the detector and server is established as shown in the Detector Installation Manual. Follow the steps below to initialize the services and control the detector.

1. To operate the detector and access its full range of features, a specialized environment has been integrated into the user server, specifically tailored for the PSS installation. To activate this environment, execute the following command in a prompt terminal:

```
$ det_env
```

The command prompt will display "(det_env)" at the beginning of the sentence, indicating that you are operating within the detector environment.

|  | **Note** |
|---|---|
| | Commands for the detector will only function within its designated environment. |

2. Start the detector camera application system (Backend). In a second prompt terminal run:

```
$ detector_backend
```

This terminal window will keep running the backend system. Open a new window to proceed to the next step.

3. Initialize software Services by opening the Command Line Interface (CLI). Run the command in the prompt terminal:

```
$ detector_console <mode>
```

There are two modes to operate the CLI: ***Common*** and ***Advanced***. Each mode enables different user levels to perform specific routines with the detector.

The common user is a limited option, often provided for operators focused on image acquisition and basic detectors' settings, with commands being simplified and image-directed. For ***Common*** use the **<mode>** argument as **empty**:

```
$ detector_console
```

The advanced user can set various internal variables, operation modes, and methods that are capable of changing the detector operation, only recommended for experienced users. For Advanced use the **<mode>** argument as **-a**:

```
$ detector_console -a
```

4. Start the camera to enable acquisitions. Run in the console:

```
> camera_start
```

# 4. Quick Start Guide

This section is intended to explain basic detector settings and procedures necessary for image acquisitions. The detector comes with a pre-defined set of basic configuration parameters, which users can load or modify as needed to customize default settings. Refer to the Advanced Routines Section to learn how to create or edit your configuration file.

Step 1 demonstrates how to load the default configuration file. If you choose this option, proceed directly to step 6. Alternatively, you can manually modify other parameters as illustrated in the subsequent steps. These steps are use-driven, for a comprehensive understanding of commands definitions, details on usage, and arguments, please refer to the following section of Commands Description.

1. **Load Configuration File:**

To configure the detector with all necessary parameters, you can utilize the command "load config INI" shown below. This command loads a file containing all required configurations, thereby preparing the detector for immediate use.

```
> load_config_ini <path>
```

The default configuration file ***config.ini*** is provided with the detector software and can be saved to any desired path by the user. It can then be passed as an argument in the command usage demonstrated above.

| | **Note** |
|---|---|
| | DACs settings may require updating over time or when operational conditions change. If necessary, please refer to the Advanced Routines Section for instructions on generating new DACs optimal settings. |

## 2. Load Equalization File:

To configure the detector, it is necessary to load an equalization matrix that is responsible for correcting the pixels response. A default file is already provided in the detector memory, which can be consulted by using the command:

```
> ls
```

This command will display all directories with equalization matrices available. The default file name can be consulted in the Validation Report Document provided.

The RAD 800K contains two modules, each with six sensors that are controlled separately. Therefore, certain operations need to be performed individually for each module. First, select the first module and complete the procedure, then select the second module and perform the same procedure. To do that use the following command:

```
> select_module 1
```

To load the existent file, in this example named ***CFG1,*** use the following command that will load the matrices for all chips of the detector:

```
> load_equalization 1 0
```

After that, select the second module to load their respective calibration, by selecting the second module and loading the second equalization, as shown below.

```
> select_module 2
```

After that

```
> load_equalization 2 0
```

| | Note |
|---|---|
| | Equalization matrices may require updating over time or when operational conditions change. If necessary, please refer to the Advanced Routines Section for instructions on generating new equalization matrices. |

**3. Set Gain Mode:**

Set a gain mode to operate the detector by running:

```
> gain_all 1
```

**4. Set Energy Thresholds**

Set the detector threshold value desired to operate (usually half of the incoming energy value). a value threshold that are half of the source energy used, using the command:

```
> threshold0_all 50
```

| | Warning |
|---|---|
| | If operating in Dual-energy mode, make sure to set the Threshold1 as well and respect TH0 < TH1. |

**Operating Energy:**

It is also possible to set the threshold in the value of energy, this applies a convention of steps of DAC into energy in KeV. This is only possible if the calibration and the values of this conversion are known and are in the rad800k.ini file. After that, use the following command:

```
> operating_energy 20
```

**5. Set Acquisition Period**

**Set the time of acquire:**

Considering the time of the x-ray source choose a time to the shutter time using the command:

```
> acquire_time 5
```

If necessary is possible to select the number of images that the detector going to take, for a simple image the number of exposures can be one as shown below.

```
> numexp 1
```

### 6. Acquire

Then use the command to realize an image acquire, name acquire, and follow by the name that you desire to save. If nothing was right, the following image will be named as image_process.hdf5

```
> acquire <file name>
```

| | Warning |
|---|---|
| | If already exist an image with the same name is in the same directory that the image going to be saved, this image will be subscript. |

## 5. Common Commands List

Once the console is up, it is possible to consult all available commands for the given mode selected (common or advanced) by running the following command. It will be displayed a list with all available commands.

```
> help
```

The most used commands for operating the detector are listed below. Note that for every command, the full description and usage mode with all possible arguments can be consulted by using the command name followed by `-h` or `–help` in the console, as follows:

```
> <command> -h
> <command> --help
```

## 1. *Energy Threshold*

**Definition:** Sets the photon energy threshold in eV. A hit will be registered in the pixel whenever the deposited energy exceeds the specified threshold.
There are two thresholds available, high and low, named `threshold0` and `threshold1` respectively. The threshold value can be applied to individual chips or to all of them. The default operation mode uses only `threshold0`, but can be changed as shown in Acquire Mode command.

**Command Usage:**

```
> threshold0_all <value>
```

**Options:**

`threshold0/threshold1`: Applies to the selected chip.
`threshold0_all/threshold1_all`: Applies to all chips in the detector.

**Arguments:**

`<value>`: Energy value in steps of DAC.

**Note**

Setting the threshold energy at half of the incoming beam energy is a common heuristic highly employed.

## 2. Operating Energy

**Definition:** Set operating energy in KeV for all chips. The energy value is usually half of the beamline energy. Then, the Threshold 0 DAC is set according to the gain mode. If no energy is passed as a param, returns the current energy.

**Command Usage:**

```
>operating_energy <enegy>
```

**Options:**

`operating_energy`: Sets the desired energy.

**Arguments:**

`<enegy>` Energy in KeV.

| Note |
|---|
| For Si sensors, it is recommended to operate with SHGM, while for CdTe sensors it is recommended to operate with HGM. The system default setting for acquisitions is SHGM. |

## 3. Select Module

**Definition:** Select or read the detector module for the next command. The modules consist of six Medipix3RX sensors arranged in a strip or array, with the capability to control each module independently.

**Command Usage:**

```
> select_module <value>
```

**Arguments:**

`<empty>`: Read the current selected module.

`<1>`: Select module 1.

`<2>`: Select module 2.

## 4. *Gain Mode*

**Definition:** Selection between four possible preamplifiers, changing the sensibility of the detector. The modes are Super High Gain Mode (SHGM), High Gain Mode (HGM), Low Gain Mode (LGM) and Super Low Gain Mode (SLGM)

**Command Usage:**

```
> gain_all <value>
```

**Options:**

`gain`: Applies to the selected chip.

`gain_all`: Applies to all chips in the detector.

**Arguments:**

`<0>`: Super High Gain Mode (SHGM)

`<1>`: High Gain Mode (HGM)

`<2>`: Low Gain Mode (LGM)

`<3>`: Super Low Gain Mode (SLGM)

**Note**
For Si sensors it is recommended to operate with SHGM, while for CdTe sensors it is recommended to operate with HGM. The system default setting for acquisitions is SHGM.

## 5. *Bias Voltage*

**Definition:** The Bias is the potential difference applied in the sensor, responsible for the charge depletion in the semiconductor crystal, making possible the diffusion of charges to come to the sensor. This command is configured exclusively for the current module. If necessary, switch to the appropriate module and reapply the command, or alternatively, use the second option by including the '–all' flag.

**Command Usage:**

```
> sensor_bias <value>
```

**Or for all modules**

```
> sensor_bias <value> --all
```

**Options:**
`sensor_bias:` Sets the bias voltage to the sensor.

**Arguments:**
`<value>` Energy value in V.

**Warning**
For Silicon detectors, the recommended operation Bias for 300 um thick sensors is 80 V, maximum value of 90 V, and for 675 um thick sensors the recommended is 120 V, maximum of 130 V. For CdTe detectors, the recommended operation bias for 1 mm thick sensors is –300 V and maximum of –500 V. These values apply to internal and external bias. Higher voltages can damage the equipment and are entirely the responsibility of the customer.

## 6. *Acquire Time*

**Definition:** Command defines the shutter time, in seconds, in which the sensor will be acquiring the images.

**Command Usage:**

```
> acquire_time <value>
```

**Options:**
`acquire_time`: Set the acquire time of images.

**Arguments:**
`<value>`: Value in seconds.

**Note**

The minimum shutter value is 8 $\mu s$ for sequential acquisition mode and 500 $\mu s$ for continuous acquisition mode.

## 7. *Acquire Period*

**Definition:** Defines the *acquisition period* of time, in seconds, in which the sensors will be acquiring and processing the image.

**Command Usage:**

```
> acquire_period <value>
```

**Options:**
`acquire_period`: Period in seconds to the acquire image.

**Arguments:**
`<value>`: Period in seconds.

**Note**

The acquire period interval is calculated by the equation:

*Acquire period = Shutter time + Readout time+ Gap Time*

Gap Time is the time between two consecutive image acquisitions and Readout Time is the time taken by Medipix3RX to process the signal, as listed in the table below.

| Readout Mode: | 12-bits Sequential | 12-bits Continuous |
|---|---|---|
| Readout Time [μs]: | 492 | 984 |

## 8. *Acquire Image*

**Definition:** The *acquire* command starts an acquisition and save of an image, the archive will be safe in the path (/home/lumentum/database/acquisitions) which the name of image_processed .hdf5 by default. The image will be safe in the HDF5 format, which allows multiple images in the same stored file.

**Command Usage:**

```
> acquire <file name>
```

**Options:**

acquire: Start the image acquisition.

**Arguments:**

<empty>: Acquire image to be saved with default name *"image_processed"*.

<file name>: Name of image acquired to be saved.

| | **Note** |
|---|---|
| | In case of files saved with the default name *"image_processed", the file is overwritten when a new acquisition is taken.* |

## 9. *Number of Exposures*

**Definition:** Defines the number of exposures, or image acquisitions that will be executed during the open shutter time in an image acquisition process. Maximum value accepted is $2^{32}$.

**Command Usage:**

```
> numexp <value>
```

**Arguments:**

<empty>: Read the current value set.

<value>: Time in seconds.

## 10. Metadata

**Definition:** Read and write custom metadata in the HDF5 image file. The following detector metadata is added automatically: shutter, period, numexp, image mode, acquisition mode, threshold0, and threshold1. However, only the custom metadata is readable.

**Command Usage:**

```
> metadata <field> <value>
```

**Options:**

metadata <field>: Read the data of the metadata field (Example: metadata name_of_field)

metadata <field> <value>: Add the field-data pair to the image.

**Arguments:**

### 11. *Save Images*

**Definition:** The *save mode* command allows configuring whether the image will be saved or not. It also has the option to save the sum of the images acquired.

**Command Usage:**

```
> save_mode <value>
```

**Options:**

`save_mode:` Choose between the different options for saving.

**Arguments:**

`<empty>:` Read the current save mode.
`<0>:` Don't save the acquire image.
`<1>:` Save the acquire image during the acquisition.
`<2>:` Save the sum of acquire images.

## 5.1. External Trigger Acquisition

To acquire images with the external trigger just follow the commands, after the connection with the backend:

```
> trigger_mode 3
```

Set the number of images desire <n>:

```
> camera_numexp <n>
```

Set the command acquire to wait the external pulse

```
> acquire
```

Send the trigger pulse.

**Note**

The camera system must be initialized previously (command 'camera_start'), and the signal of pulse must start after the acquire command.

## 5.2.Image Visualization

The generated data can be visualized using some different applications, the Silx Viewer as described below.

- **Detector Vis**

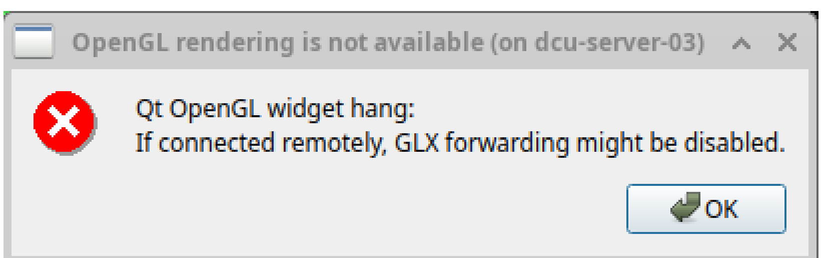
  The *Detector Vis* is real time visualizer that works with the backend based on the *Silx View*, with a maximum frame of 30 fps in real time. To use the visualizer, open a new terminal and enter the detector environment:

```
$ det_env
```

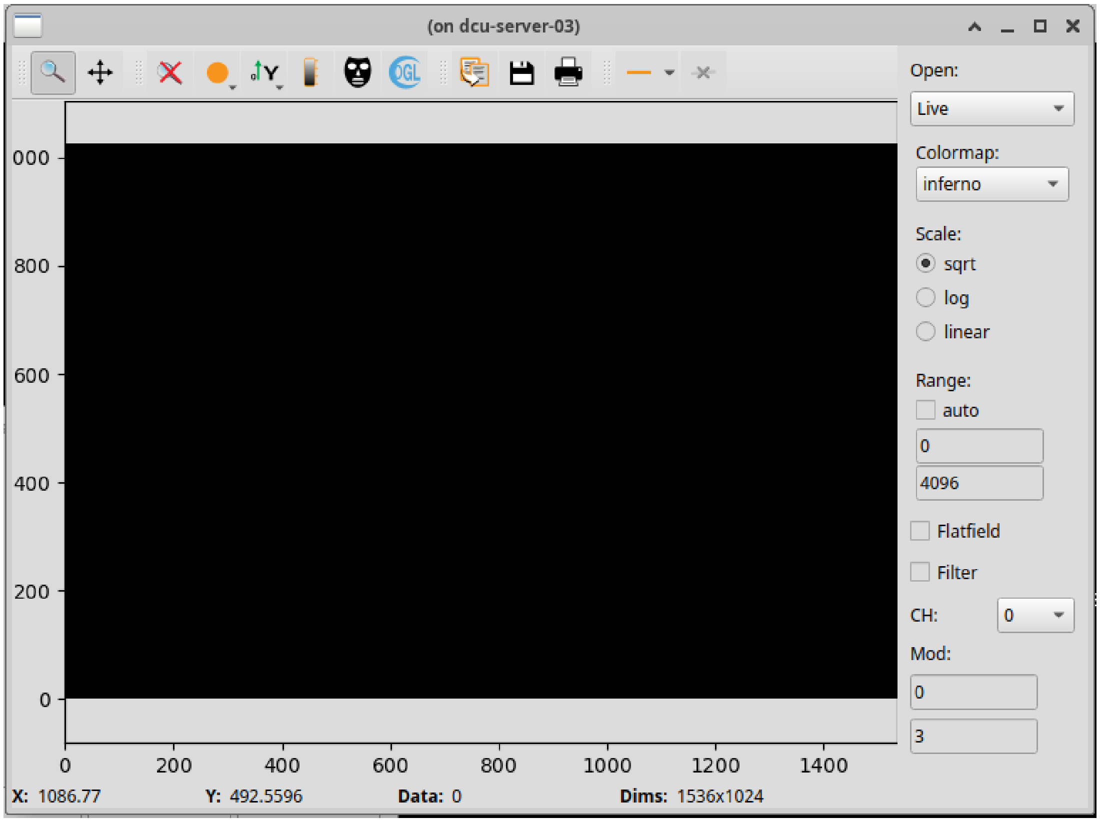
  After that use the command:

```
$ detector_vis
```

  A window will appear, click in ok



Then the window of the visualizer will open as shown below:




The image only updates after the closer of the frame.

- **Silx Viewer**

The Silx Viewer is a software tool designed for visualizing and analyzing scientific data, particularly can be used to visualize X-ray images. The software supports visualizing various types of scientific data, including 1D, 2D, and 3D datasets, and read the format HDF5 used.

The software offers a range of analysis tools tailored to scientific data, including data fitting, peak finding, integration, and more, enabling them to extract meaningful information from their datasets. They are also designed to run on multiple operating systems, including Windows, macOS, and Linux, ensuring accessibility to a wide range of users.

Overall, the Silx Viewer provides a comprehensive solution for visualizing and analyzing scientific data in the field of X-ray, offering intuitive interfaces, powerful analysis capabilities, and flexibility for customization and integration with other tools.

After mounting the detector database at the local computer, the images can be opened in the terminal using Silx:

```
$ silx view <image-path>
```

Or to open all hdf5 files:

```
$ silx view <*.hdf5>
```

It's possible as well to open multiple images at once using the command:

```
$ silx view <image-path-1> <image-path-2>
```

After that to visualize the image is just double click in the left column where is right *entry,* that going to open a second item called *data,* double click in data and a third index will open called also *data,* after that is possible to see the image as a matrix, for visualize as an image just click in *image* in the menu below.

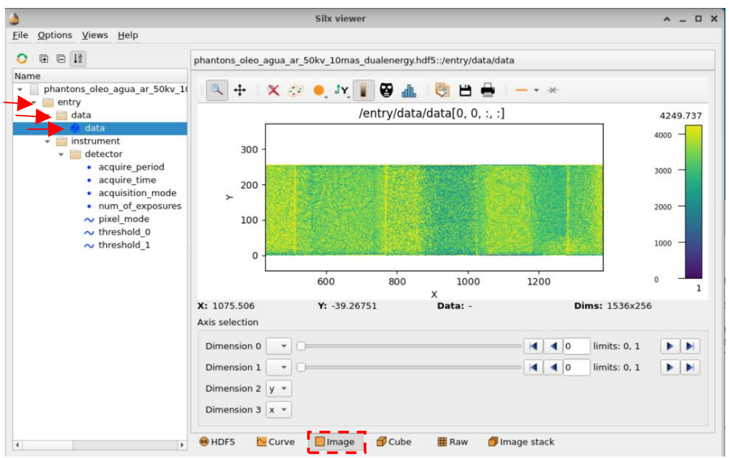



**Figure 2:** Image organization.

The sequential and the dual energy images can be visualized by clicking on the arrows below the visualizer in the Silx software.

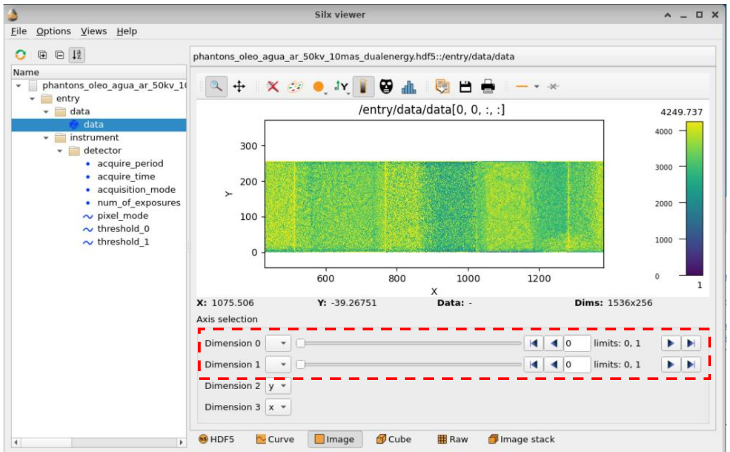



**Figure 3:** Selection of multiple images.

# 6. Advanced Controls

## 6.1. Advanced Commands List

### 1. Load Equalization

**Definition:** This command loads the matrix of equalization of the SD card in the selected chips, correcting the natural difference in the sensitivity to the incoming radiation. The files can be consulted using `ls`.

**Command Use:**

```
>load_equalization <matrix> <chips>
```

**Options:**
`Load_equalization`: load the equalization matrix into the chips

**Arguments:**
`<1,2,3,…,n>:` Choose the Matrix to load. The file is named CFG<n> where only the number n of the matrix is needed.
`<1,2,3,…,n>:` Choose which chip to use, 0 select all available chips. (Comma-separated values, without space).

### 2. *Acquisition Mode*

**Definition:** The detector has four modes of operation, which change the way that the sensor registers the data for different possibilities of use. They are the sequential 12bits, the sequential 24bits, dual energy, and continues.

- **Sequential 12 bits:** In this mode, the detector uses only the low discriminator and the low 12-bit counter for image acquisition. In this mode, pixel counting is limited to 4095, after which further events are disregarded.
- **Sequential 24 bits:** This mode is the dynamic range mode that uses both the low and high discriminators, as well as the 12-bit counters, to acquire images. In this mode, the number of pixels counted is limited to 167772155, after which additional events are ignored. This acquisition mode requires a readout time of 984 µs per image.
- **Continuous:** Using both the low and high discriminators and the 12-bit counters for image acquisition, the continuous mode can make images without stops. In this mode, two images are obtained, one from the low counter and the other from the high counter, one after the other. While the low counter is acquiring, the high counter is executing the readout process and waiting for its next acquisition. Then, the high counter starts acquiring while the low one does the readout.
- **Dual Energy:** In the dual-energy mode, the detector uses both the low and high discriminators and the 12-bit counters working independently in the same window of acquisition to generate 2 images, the first from the low counter and the second from the high one. Each image represents an energy threshold. This mode has to have the low counter threshold lower than the high. In this mode, a readout time of 984 µs is required.

**Note**

For operating with continuous (2x12-bit), it is required to have the acquire time bigger than readout and number of exposures must be bigger than 2.

### 3. *Trigger Mode*

**Definition:** The trigger mode is a command that calls the acquire of an image, this can be programmed to execute different patterns. The detector can be triggered by software (internal trigger) or by an externally applied trigger signal (external trigger).

**Command Usage:**

```
> trigger_mode <mode>
```

**Options:**
<0>: Cascade disabled, an internal trigger for each module (Default)
<1>: Internal trigger from master module, trigger output set by shutter time
<2>: Internal trigger from master module, trigger output set by acquisition time
<3>: External trigger with positive edge at master module, trigger output set by shutter time
<4>: External trigger with positive edge at master module, trigger output set by the acquisition time

### 4. *Reset*

**Definition:** The command reset and restore the parameters by default t.

**Command Usage:**

```
> reset
```

**Options:**
reset: execute the reset command.

**Arguments:**

### 5. *Dac Scan*

**Definition:** The command dac_scan starts the acquisition of the relation voltage by the DAC steps showing the behavior from which DAC, by default the scan is made in the GND, FBK, and CAS DACs to find the optimal value of the DACs that are GND = 650 mV, FBK = 900 mV and CAS = 850 mV. All the DACs can be found in the help command.

**Command Usage:**

```
> dac_scan <argument>
```

**Options:**
`dac_scan`: The DAC's name to be analyzed.

**Arguments:**
`<empty>`: Start the Scan of the DAC bias.

`<1,2,3,…,n>:` Choose which chip to use, 0 select all available chips. (Comma-separated values, without space).
`<info>:` Display related parameters information.
`dac <dac name>:` Define the DASs to be scanned.
`<dac> N:` Set the dac voltage setpoint. (Except Band_Gap_Output, Band_Gap_Temperature, DAC_Bias, DAC_cascode_bias)
`load_dac N:` Load the DSC target value (0 for disable, 1 for enable).
`star N:` Define the initial value from the dac scan.
`step N:` Define the step value to the dac scan.
`stop N:` Define the stop value to the dac scan.
`title N:` Change the folder name where the file will be saved.
`verbose N:` Display scan values on screen (0 for disable, 1 for enable).

## 6. *Threshold Scan*

**Definition:** The *threshold scan* starts the values scanning of the energy photon discriminators to find the relation of energy from an x-ray source which steps values, this is made while acquiring an image to each step, creating a graph of counts by DAC steps.

**Command Usage:**

```
> threshold_scan <parameters>
```

**Options:**
Threshold_scan: Start the Threshold Scan after showing the set parameter.

**Arguments:**
`<empty>`: Execute threshold scan.
`<1,2,3,…,n>:` Choose which chip to use, 0 select all available chips. (Comma-separated values, without space).
`<info>:` Display related parameters information.
`scan_th N`: Chose which DAC threshold energy to use. Range: 0 to 7.
`star N:` Define the initial value from the dac scan.
`step N:` Define the step value to the dac scan.
`stop N:` Define the stop value to the dac scan.
`title N:` Change the folder name where the file will be saved.

| | Note |
|---|---|
| | The camera system must be initialized previously (command 'camera_start'). |

## 7. *Equalize*

**Definition:** The command equalize realizes the equalization of the sensor. The MADIPIX sensor has a circuit for which pixel, and due to component fabrication differential, which pixel reads a different value to the same photon. This made the photon average response more equal to all pixels Changing the DACs DiscL/H to adjust and ConfigDiscL/H to the fine toning.

**Command Usage:**

```
> equalize <parameters>
```

**Options:**
threshold_scan: Star the threshold scan after showing the set parameters.

**Arguments:**
<empty>: Execute equalization procedures.
<1,2,3,…,n>: Choose which chip to use, 0 select all available chips. (Comma-separated values, without space).

<info>: Display related parameters information.

th_target N: Noise, the equalization is made when the electronic noise floor from the chips or Tp (test pulse), equalized by the circuit of test pulse, where is possible to set a current to equalize.

dac_scan N: Defines if targets will be set for DACs

Dac <dac>: Define the DACs to be scanned.

<dac> N: Set the dac voltage setpoint. (Except Band_Gap_Output, Band_Gap_Temperature, DAC_Bias, DAC_cascode_bias) scan_th N: Chose which DAC threshold energy to use. Range: 0 to 7.
scan_th N: Chose which DACs threshold energy to use. Range: 0 to 7.
Configdiscl N: Define ConfigDiscL initiation values. Range: 0 to 31.
Configdisch N: Define ConfigDiscH initiation values. Range: 0 to 31.
Optimal_disclh N: Enable optimal DiscL/H scans execution. (star = 0, step =1, stop = 120) and process the data. (0 for disable, 1 for enable).
star N: Define the initial value from the dac scan.

step N: Define the step value to the dac scan.

stop N: Define the stop value to the dac scan.

title N: Change the folder name where the file will be saved.

load_matrix N: Send equalization matrices to SD Card and load them on the selected chips (0 for disable, 1 for enable).

erase_matrix N: Delete equalization matrices from SD Card (0 for disable, 1 for enable).

Th_scan_after N: Execute the threshold scan after equalization (0 for disable, 1 for enable).

**Note**

The camera system must be initialized previously (command 'camera_start').

## 8. *Chip Selection*

**Definition:** Select which chip will be sending or receiving commands to.

**Command Usage:**

```
> imgchip_id <chip>
```

**Options:**
imgchip_id: select a chip to use.

**Arguments:**
<chip>: number of the chip chosen, star in 1.

## 9. *DAC Get*

**Definition:** Read all digital and analog DAC values of the selected chip.

**Command Usage:**

```
> DAC_get_all <Name of the safe file>
```

**Options:**

`DAC_get`: select a DAC to read.
`DAC_get_all`: Read all digital and analog DAC values for all the chips and save the result in a file.

**Arguments:**
`<name>`: Name of the file containing the DAC values to all chips.

## 6.2.Advanced Routines

### 6.2.1. Equalization

Due to the mismatch of the transistors caused by irregularity in the fabrication, this creates a dispersion in the discriminator threshold. To minimize this difference, a local correction of this voltage is applied at each pixel to compensate for the dispersion of electrical characteristics between pixels by making two adjustments, a fine, and a coarse tuning. The fine-tuning operates local-adjust made to control a 5-bit current DAC (ConfigDiscL and ConfigDiscH) (Figure 4) that can add or subtract current from the input signal, depending on the selected digital code. The coarse tuning operates in the dynamic range of the current DACs in the chip, which can be set by the user using an 8-bit DAC (DiscL and DiscH). This adjustment process is called equalization.

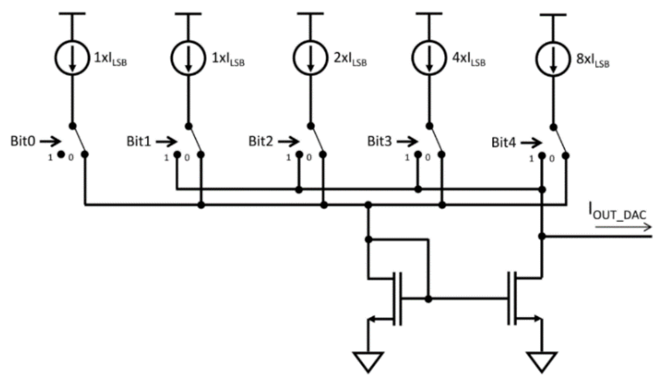



**Figure 4:** 5-bit DAC on each discriminator.[1]

The equalization procedure can be executed using different input signals. The simplest and fastest is by electronic noise floor that uses the equalization mode, in which the Medipix3RX uses the electronic noise to simulate incoming X-ray photons. This input mode delivers a satisfactory performance for low energies, but it decreases for medium and high energies.

Another way to duet is by x-ray fluorescence using an external x-ray source that gets optically sensed and then converted to electronic pulses by the Medipix3RX chip – this mode is the most encompassing and precise input mode, and it delivers the highest performance since it considers the sensor layer. This equalization mode is under development.

Threshold calibration is very stable in time and so needs not be redone very often, once a year is normally sufficient. It will provide an equalization for 12-bit SPM mode that can be loaded by the Command-Line Interface, as shown in the quick start guide section. . To use this procedure, it is necessary to enter each individual module using:

First you must be in the detector environment:

```
$ det_env
```

After that enter the console of the desire module <n>:

```
$ detector_console -a -m<n>
```

If the backend isn't open, open a new detector backend:

```
$ detector_backend
```

Go back to the console and the connection with the backend:

```
> camera_start
```

After that use the command of equalization:

```
> equalize
```

That will show the parameters info that you can check, after that press enter to confirm and execute the procedure.

**Warning**

If this procedure applies to only a single sensor module, it is necessary to select each module individually and create an equalization matrix for each module to equalize all chips.

### 6.2.2. Threshold Scan

Part of the process of equalization is the threshold scan, this procedure shows the behavior of the sensor before the equalization. To do that use the Medipix3RX discriminators, this discriminator assignee an energy threshold value (i.e., threshold [0:7] DACs) that is matched against the sensed photon energy to determine whether to count that photon or not — a photon is counted if its energy is equal to or higher than the set energy threshold.

The Threshold Scan procedure scans a threshold DAC while acquiring an image on each threshold step. As an outcome, a tree graph is made, and two of the sums of counts per chip (Fig. 5) as a function of threshold steps are produced. This allows the algorithm to find the distribution caricaturists of the noise-floor natural of the hardware.

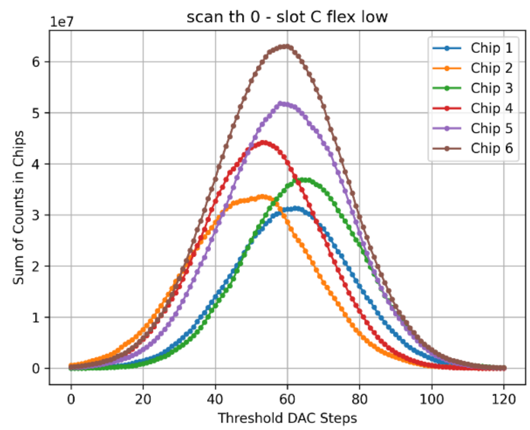



**Figure 5:** Example of threshold scan.

After that, a third graph that shows this noise shifted below a DAC step range, not allowing this noise to affect the final image (Fig. 6), if the threshold0 was used above this range.

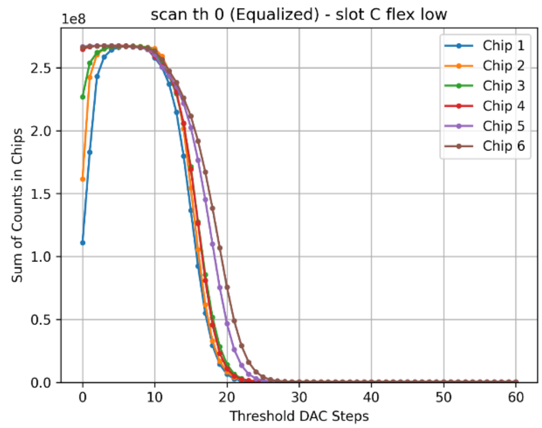



**Figure 6:** Example of threshold scan after.

It can be executed using the command:

```
> threshold_scan
```

That will show the parameters info that you can check, after that press enter to confirm and execute the procedure.

| | **Warning** |
|---|---|
| ⚠ | If this procedure applies to only a single sensor module, it is necessary to select each module individually and create an equalization matrix for each module to equalize all chips. |

### 6.2.3. Dac Scan

To better equalize usability, the different DACs must be in specific mV values, due to the fabrication process, these values are obtained in different DAC steps, these are called optimal values and are valid for the GND, FBK, and CAS DACS. To find this optimal value, a scan is made of each DAC, creating a graph of DAC step by Voltage (Fig. 7). All other DACs must be monitored as well as DAC outputs in the same range as the Medipix3RX manual. The target values are GND = 650mV, FBK = 900mV, and CAS = 850mV. These values can be set up using the tracking features.

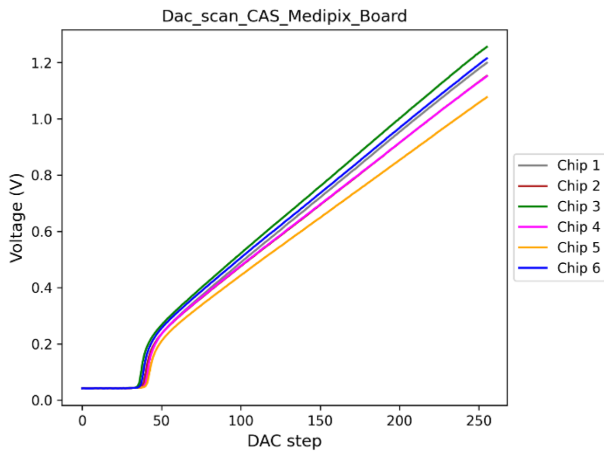



**Figure 7:** Example of DAC curve.

| | **Warning** |
|---|---|
| ⚠ | If this procedure applies to only a single sensor module, it is necessary to select each module individually and create an equalization matrix for each module to equalize all chips. |

### 6.2.4. Flat Field

The pixels of an area detector have different sensitivities to incoming X-rays and will have slightly different counting efficiencies concerning each other. To obtain high-quality measurements, the detector must be calibrated to correct for these inhomogeneities in the pixel response. This is a flat field, which can be achieved by illuminating the entire active area of the detector with homogeneous radiation and accumulating sufficiently large statistics to be sensitive to the required calibration level.
To use the flat field, it is necessary to create a flatfield file, after which that is loaded into the detector by the command.

```
>flat_field_filepath <path>
```

After loading the file, to use the flatfield in the images use the command:

```
>frame_process_mode 1
```

### 6.2.5. Config_ini file creation

To prepare the detector, the easy way is by using the load_config_ini, this command sets some parameters as the DAC values, the thresholds low and high, the gain, the bias, the equalization, and the equation with the values to calibrate the detector to energy values.

That is interesting to be able to change these values if necessary, making it possible to operate the detector more effectively. To create a new file, copy the initial rad800k.ini file and edit the file changing parameters like the threshold0 and threshold1, the bias, the gain, and the values of the energy calibration.

after that use the command:

```
>load_config_ini <new file>
```

## 6.3.DACs

The Medipix3RX chips have 31 DACs, with 27 that can be set and read, and 4 read only. From these 27 DACs, some of them have an impact on the image acquisition and the detector operation and are available to be set, if required. These DACs are listed below with the nominal value found in the Medipix3RX manual:

**Table 1:** Biasing signal for the principal DACs.[1]

| DAC | Nominal Value |
|---|---|
| **I_Preamp** | 1.5µA |
| **I_Krum** | 1nA |
| **V_Fbk** | 800mV (600mV<V_Fbk<900mV) |
| **V_Cas** | 800mV |
| **V_Gnd** | 600mV (550mV<V_Gnd<(V_Cas – 152 mV)) |
| **I_Shaper** | 0.55µA |
| **V_Rpz** | 1.5V(OFF) |
| **I_Disc** | 1µA |
| **I_Disc_LS** | 0.35µA |
| **I_DAC_DiscL** | 10nA |
| **I_DAC_DiscH** | 10nA |

To find the values of with DAC use the command DAC_get as describe in the advanced commands list section.

**OMR:**

| | Note |
|---|---|
|  | These settings should be adjusted only for advanced users, otherwise, it may cause the improper behavior of the detector. |

## 6.4. Trigger Commands

### 6.4.1. Trigger Modes

RAD has one external trigger input, TRIG IN and one processed trigger output, TRIG OUT, which provide different hardware signals from the FPGA to synchronize with other equipment. Figure 8 illustrates the external trigger system's electrical interfaces and connections.

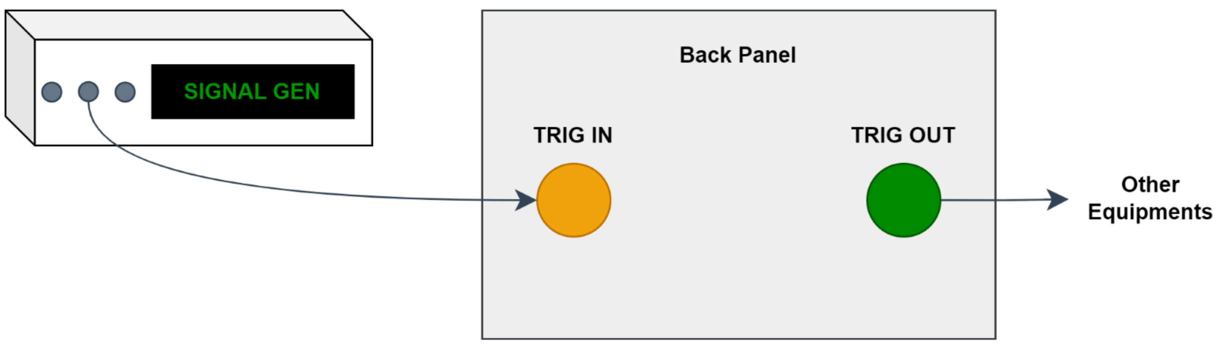



**Figure 8**: Schematic of external trigger electrical interfaces.

The desired trigger mode can be selected using the terminal command trigger_mode, as described in Table 2.

**Table 2**: Trigger mode parameters.

| Arguments | Description |
|---|---|
| 0 | Cascade disabled, an internal trigger for each module (Default). |
| 1 | Cascade enabled, internal trigger from master module, trigger output set by shutter time. |

| | |
|---|---|
| 2 | Cascade enabled, internal trigger from master module, trigger output set by acquisition time. |
| 3 | Cascade enabled, external trigger with positive edge at master module, trigger output set by shutter time. |
| 4 | Cascade enabled, external trigger with positive edge at master module, trigger output set by acquisition time. |
| empty | Read the trigger input and output for all the modules. |

### 6.4.2. Internal Trigger (Default)

Image acquisition starts with the software trigger only. The acquire command is used to start acquisitions and follows the user-defined parameters. The processed output remains disabled and no signal is available in TRIG OUT.

```
$ detector_console -d
RAD800k» numexp 1000
Number of Exposures set 1000
RAD800k» acquire_time 0.005
AcquireTime set with 0.005 seconds
Shutter time: 0:00:00.0050000
RAD800k» trigger_mode 0
0
RAD800k» acquire 1
Start acquisition
```

### 6.4.3. Internal Trigger with processed output following the shutter opening

Image acquisition starts with the software trigger only. The acquire command is used to start acquisitions. The processed output fires an active LOW signal with the same time width of the shutter exposure.

| | Note |
|---|---|
| | It's important to see that the idle state of the shutter signal is inverted, so the open shutter (acquisition time) is represented by low state. |

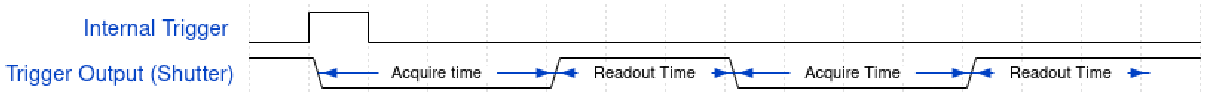



**Figure 9**: Trigger Mode 1 considering two acquisitions.

### 6.4.4. Internal Trigger with processed output following the acquisition

Image acquisition starts with the software trigger only. The acquire command is used to start acquisitions. The processed output fires an active HIGH signal while the detector is running an acquisition. In terms of sensor's operations, an acquisition has the duration of the acquire period (shutter exposure + data readout + gap between frames, if set) times the number of frames desired.

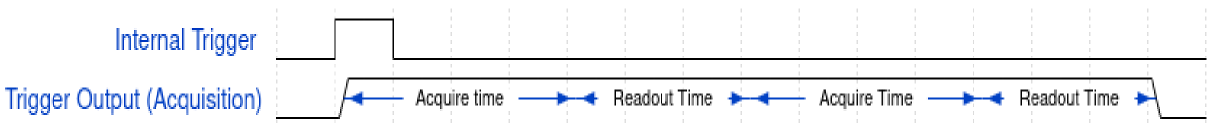



**Figure 10**: Trigger Mode 2 considering two acquisitions.

### 6.4.5. External Trigger

In both modes of external trigger available – with processed output following the shutter opening (mode 3) or following the acquisition (mode 4) –, the image acquisition starts as soon as an electrical signal with rising or falling edge is detected in the TRIG IN input.

As seen in Section 1.1, when using external trigger, the det_numexp command set the number of acquisitions per trigger signal and camera_numexp defines the total number of acquisitions that will be saved. To save all images, camera_numexp must be equal to *camera_numexp = total of trigger signals* × *det_numexp*. Use camera_acquire command to begin an acquisition.

| | **Note** |
|---|---|
| | It is recommended that *det_numexp* be set to 1 because any trigger signals sent during an acquisition will be ignored. If *det_numexp* is greater than 1, please set the trigger signals period to be greater than the total period for all acquisitions. |

### 6.4.6. Advanced Trigger Configuration

The RAD detector supports event trigger synchronization across devices. This feature allows an internal or external event to initiate the detector's image acquisition process. It also enables the detector to transmit trigger events to other devices. check out the User Manual of detector.

#### 6.4.6.1. Trigger Advanced Commands

Similar to common user command, the trigger_mode selects the desired trigger mode. When the argument is set to 0, the advanced user manually configures the external trigger input using the advanced command ext_trigger and the processed output trigger using  trigger_out (Table 3).

**Table 3**: RAD supported trigger modes and configuration.

| Trigger Mode | Description | Configuration |
|---|---|---|
| **0** | Cascade disabled, ext_trigger ‘0’ an internal trigger for each module (Default). | ext_trigger ‘0’<br>trigger_out ‘0’ |
| **1** | Cascade enabled, internal trigger from master module, trigger output set by shutter time. | ext_trigger ‘0’<br>trigger_out ‘10’ |
| **2** | Cascade enabled, internal trigger from master module,trigger output set by acquisition time. | ext_trigger ‘0’<br>trigger_out ‘14’ |
| **3** | Cascade enabled, external trigger with positive edge at master module, trigger output set by shutter time. | ext_trigger ‘1’<br>trigger_out ‘10’ |
| **4** | Cascade enabled, external trigger with positive edge at master module, trigger output set by acquisition time. | ext_trigger ‘1’<br>trigger_out ‘14’ |

This command can set only predefined modes, however the advanced user can set each parameter of trigger subsystem and generates any combination.

**External Trigger**

The ext_trigger defines how the acquisition is controlled by trigger, initializing or finishing the acquisition considering the edge sensitivity.

**Table 4**: ext_trigger available options.

| Mode | Description |
|---|---|
| **0** | External disabled, only internal trigger. |
| **1** | Start with positive edge (Positive Simple). |
| **2** | Start with negative edge. (Negative Simple) |
| **3** | Start with positive edge and finish with positive edge (Positive Start and Stop). |
| **4** | Start with positive edge and finish with negative edge (Positive Gate) |
| **5** | Start with negative edge and finish with negative edge (Negative Start and Stop). |
| **6** | Start with negative edge and finish with positive edge (Negative Gate) |

All available modes are described below.

- Simple mode: All software definitions will control the image, as exposure time, number of images and frequency of frames per second.

| | |
|---|---|
|  | **Warning**<br>If stop is activated for advanced modes, the user **should** define the maximum exposure time. The recommended value is 18446744073709 seconds, but a security value greater than trigger period is enough. This exposure time can finish the acquisition before the stop signal, in this way the signal is discarded. |

- Start and Stop mode: The start and stop advanced mode starts and finishes the acquisition for each edge of external trigger signal.

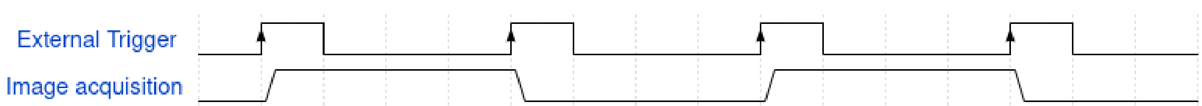



**Figure 11:** Positive Start and Stop trigger mode.

- Gate mode: For this mode, the image acquisition or shutter control follow the external trigger signal.

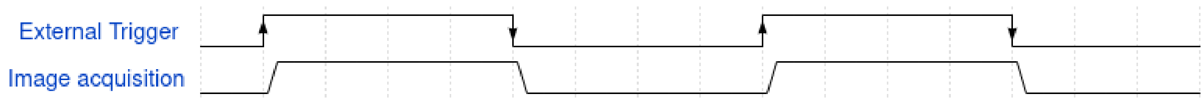



**Figure 12:** Positive Gate trigger mode.

**Processed Trigger Output**

The trigger_out is responsible for processed trigger output configuration.

**Note**

The idle state of the shutter signal is inverted, so the open shutter is represented by low state.

**Table 5**: trigger_out available options.

| Mode | Description |
|---|---|
| **0** | Disable |
| **1** | Internal by Software |
| **2** | External Input |
| **3** | SW or External Input (Enable both modes) |
| **4** | Start Trigger |
| **5** | Stop Trigger (If stop is used) |
| **6** | Start/Stop Trigger (If stop is used) |
| **7** | Gate Mode |
| **8** | Shutter MB slot-A |
| **9** | Shutter MB slot-B |
| **10** | Shutter MB slot-C (For light detector) |
| **11** | Shutter MB slot-A, B or C (Enable shutter for all MB’s) |
| **12** | Acquisition MB slot-A |
| **13** | Acquisition MB slot-B |
| **14** | Acquisition MB slot-C (For light detector) |
| **15** | Acquisition MB slot-A, B or C (Enable acquisition for all MB’s) |

**Trigger Control**

There is a feature of triggering system to control the number of external trigger pulses that can initialize an acquisition, called *NumTrigger* (Number of Triggers).

The num_ext_trigger receives the number of external trigger pulses to be accepted, any following input signal is discarded and the acquisition isn't initialized. To disable the feature, it's necessary set the numtrigger as zero: num_ext_trigger 0. For example, if the user defines the numtrigger as 3, the pulses after the third are discarded.

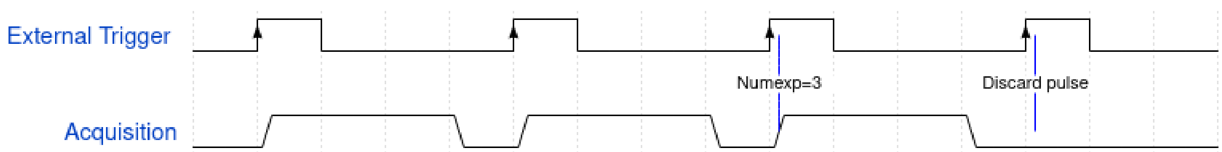



**Figure 13:** Numtrigger set as 3 discarding the fourth pulse.

To receive the external trigger again, the numtrigger feature must be disabled:

**Note**

If it's desired that the numtrigger limits a new number, the command should be done with '0' to deact ivate and then with the new number of triggers to be accepted.

```
$ detector_console -a
RAD<det model>» numexp 1
Number of Exposures set 1
RAD<det model>» num_ext_trigger 3
Numtrigger enabled
RAD<det model>» acquire
Start acquisition


$ detector_console -a
RAD<det model>» num_ext_trigger 0
Numtrigger disabled


$ detector_console -a
RAD<det model>» numexp 1
```

```
Number of Exposures set 1

RAD<det model>» num_ext_trigger 3
Numtrigger enabled

RAD<det model>» acquire
Start acquisition


$ detector_console -a

RAD<det model>» num_ext_trigger 0
Numtrigger disabled
```

# References:

[1] Ballabriga, R., & Llopart, X. (2012). Medipix3RX manual. *Medipix3RX manual*.

## Stamp and Signature: